Slice+Slice Baby: Generating Last-Level Cache Eviction Sets in the Blink of an Eye

Bradley Morgan*†, Gal Horowitz‡, Sioli O'Connell*, Stephan van Schaik§, Chitchanok Chuengsatiansup¶
Daniel Genkin‖, Olaf Maennel*, Paul Montague†, Eyal Ronen‡ and Yuval Yarom**
*The University of Adelaide †Defence Science and Technology Group
‡Tel-Aviv University §University of Michigan ¶The University of Klagenfurt
‖Georgia Tech **Ruhr University Bochum

arXiv:2504.11208v2 [cs.CR] 20 Apr 2025

***Abstract*—An essential step for mounting cache attacks is finding eviction sets, collections of memory locations that contend on cache space. On Intel processors, one of the main challenges for identifying contending addresses is the sliced cache design, where the processor hashes the physical address to determine where in the cache a memory location is stored. While past works have demonstrated that the hash function can be reversed, they also showed that it depends on physical address bits that the adversary does not know.**

In this work, we make three main contributions to the art of finding eviction sets. We first exploit microarchitectural races to compare memory access times and identify the cache slice to which an address maps. We then use the known hash function to both reduce the error rate in our slice identification method and to reduce the work by extrapolating slice mappings to untested memory addresses. Finally, we show how to propagate information on eviction sets across different page offsets for the hitherto unexplored case of non-linear hash functions.

Our contributions allow for entire LLC eviction set generation in 0.7 seconds on the Intel i7-9850H and 1.6 seconds on the i9-10900K, both using non-linear functions. This represents a significant improvement compared to state-of-the-art techniques taking 9× and 10× longer, respectively.

1. Introduction

In the two decades since their introduction [38, 40, 49, 50], cache attacks have been recognised as a significant security threat. Multiple attacks have been published, leaking secret or sensitive information from a wide range of applications, including cryptography [13, 14, 32, 58, 60], user interface [13, 36, 46], and others [23, 37, 59, 63]. Moreover, in recent years, cache attack techniques are being used as stepping stones for more advanced attacks [12, 30, 31].

A fundamental prerequisite for many contention-based cache attacks is to find an *eviction set*, a collection of memory addresses that contend on cache space with a victim memory address. In a typical Prime+Probe attack [32, 38], the eviction set serves two purposes. First, accessing the eviction set creates contention, forcing eviction of the victim memory address. Second, by measuring the access time to memory addresses in the eviction set, the attacker can determine whether they are cached and from that infer whether the victim has accessed the victim memory address.

The main challenge for constructing eviction sets is *addressing uncertainty* [56], which hides cache addressing information from the attacker. Standard threat models involve attackers executing unprivileged user-level code under virtual memory addressing. Conversely, caches typically use the physical memory address for determining the cache location in which memory is stored. The virtual-to-physical address mapping hides the physical address from the attacker, hampering direct determination of cache location from virtual addresses.

Early works overcome addressing uncertainty by either targeting the L1 cache [38, 40], where the address bits that determine the cache location are preserved by the virtual to physical address translation or by using huge pages to ensure that more address bits are preserved [21]. However, the sliced design of the Intel last-level cache (LLC) exacerbates addressing uncertainty and precludes such techniques from working for this cache on more recent processors. The Intel sliced cache is partitioned into multiple slices, each serving a different part of the physical address space. To determine the slice that serves a memory address, the processor computes an undisclosed hash function based on the physical address bits. Past efforts for reverse engineering the slice function have demonstrated that it uses all of the address bits [15, 22, 33, 34, 61]. Consequently, without knowing the full physical address, the adversary cannot determine the slice to which a memory location maps.

Liu et al. [32] propose to exploit cache misses due to contention in order to construct eviction sets. This approach has been improved through a series of algorithmic [28, 47, 52, 57] and technical [42, 48, 64] improvements. Due to their importance for cache attacks, the difficulty of constructing eviction sets is considered a measure for the security of defences against side-channel attacks [9, 45, 55]. In response, some designs that are capable of finding eviction sets in the presence of side-channel countermeasures have been proposed [26, 41, 43].

Most proposed approaches for eviction-set creation exploit the observation that when the number of slices is a power of two, eviction sets at different page offsets are equivalent [32, 61]. Thus, on such machines, the attacker only needs to find eviction sets for a single page offset

and can directly propagate that information to all other page offsets in the page. This approach does not work for machines where the number of slices is not a power of two and the slice function is not linear [61]. Nonetheless, Yarom et al. [61] demonstrate that for a six-core machine, it may be possible to propagate information from some page offsets to the rest of the page.

1.1. Our Contribution

In this work, we present further improvements to the art of eviction set construction. For that we build on two main advancements. First, we show how to use timing variations to determine the slice a memory address maps to. This information allows us to further partition the initial candidate set memory, achieving additional speedup. Secondly, we show how to use properties of the known hash function to carry over information across page offsets, allowing us to recover eviction sets for the whole address space while only probing a fraction of the cache lines. We now describe our algorithm in further detail.

As a first step for our algorithm, we develop a technique to identify the cache slice that a memory location maps to by exploiting differences in latency that each slice exhibits due to the processor's physical layout. The main challenge is that due to system load and changes in processor frequency, the latency of slice accesses vary and overlap with one another, making it difficult to narrow down the slice that a memory location maps to. To overcome this challenge, we build on microarchitectural weird machines [7, 14, 24, 26, 54], constructing a gadget which uses a memory access race condition to determine that with the lower access time. We refer to this as the comparator gate, and observe that because system load impacts memory accesses similarly, comparing these helps lessen the effects of noise. Using our comparator gate, we determine the slice that a memory location maps to with an average accuracy of 97% from any processor core.

Then, we propose an approach to improve the speed of our algorithm by extrapolating slice mappings from a few recovered memory addresses to other addresses in the same page. Specifically, we exploit the observation that the processor's hash function generates only a few possible mappings from virtual page offsets to slices. Consequently, by recovering a few locations in a virtual page, we can determine its overall slice mapping and propagate this to the rest of the locations in the page, greatly speeding up an eviction set construction. Moreover, this technique also allows for error detection, because in the case of error, there is a high probability that the recovered slice numbers do not match any of the possible patterns.

To construct eviction sets, we generate a set of candidate addresses, all of which reside in the same page offset. We partition this set, first based on the L2 cache set [64] and secondly based on the LLC slice as determined by our comparator gate and slice mapping propagation method. We then use a conflict-based eviction-set construction algorithm on each partition separately.

As a final contribution, we show how to exploit the knowledge about the hash function used for mapping memory into slices to propagate eviction set information on non-linear slice functions. Specifically, we demonstrate that by constructing eviction sets in 4 KB page offsets, we can identify working eviction sets in other offsets, requiring only 15–22% of the total eviction sets to be built conventionally, depending on the function. This achieves a total execution time improvement of 8× to 10× over prior work. For machines with linear slice functions, our algorithm is 1.1× to 1.5× faster than state-of-the-art techniques.

In summary, the contributions of this work are:

- We demonstrate a new weird gate construction that can identify the LLC slice that a memory location maps to (Section 4).
- We show how to use the known hash function to detect and correct errors in slice mapping as well as to propagate slice information across pages (Section 5).
- We demonstrate how to propagate eviction-set information across page offsets when the processor uses a non-linear hash function (Section 6).

Following the practices of open science, we make our source code and research artifacts publicly available at https://github.com/0xADE1A1DE/Slice-Slice-Baby.

2. Background and Related Work

In this section, we introduce the necessary background on Intel caches, their LLC slice function and past recovery efforts, eviction sets and microarchitectural weird gates.

2.1. Intel Cache Hierarchy

Processor caches store sections of main memory to reduce access time. A cache miss occurs when the requested data is absent in the cache, whereas a hit occurs when the data is found. Modern caches are typically n-way set-associative, split into m-sets with specific indexing from memory address bits. Each Intel CPU core has its own L1 data and instruction caches, and also an L2 cache. The last-level cache (LLC) is the largest, shared across cores and located in the uncore section of the processor die [17], containing all non-CPU core logic. Accessing memory loads a 64-byte chunk into the L1 cache as a cache line, with evictions occurring in accordance with replacement policies from L1 to L2 and beyond. Older Intel CPUs feature an *inclusive* LLC, holding copies of L1 and L2 data. Modern processors use *non-inclusive* caches [18], where data held in the private caches may or may not appear in the LLC. Here, a coherency directory (CD) or snoop filter (SF) is used to maintain cache coherency without storing duplicate data from lower-level caches [17, 20]. These mechanisms track the state and location of cached data across cores and have a similar set-associative structure as the caches [58, 64]. For our work, we focus on several Intel processors with inclusive and non-inclusive LLCs. Table 1 shows the cache parameters for such processors.

TABLE 1: Intel cache parameters with m-sets and n-ways.

Processor	L1 Data		L1 Instr.		L2		LLC / Slice	
	m	n	m	n	m	n	m	n
i7-6700K, i7-8700 i7-9850H, i9-10900K	64	8	64	8	1024	4	1024	16
i7-11700K	64	12	64	8	1024	8	1024	16
i7-12900KF (P-Core) i7-13700H (P-Core)	64	12	64	8	2048	10	4096	12
i9-14900K (P-Core)	64	12	64	8	2048	16	4096	12

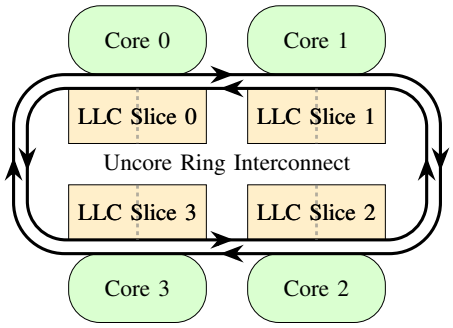

Figure 1: Logical structure of the ring interconnect on Intel Core CPUs. It allows for the bi-directional data transfer [4, 39] between cores, slices/sub-slices and other structures.

2.2. Sliced Caches

Intel's design of the LLC splits it into several distinct *slices*, separately connected to the processor's interconnect in the uncore region of the processor [4, 19, 20, 39]. Memory maps to certain slices based on the physical address according to a proprietary hash function undisclosed by Intel. Based on prior reverse engineering efforts [15, 16, 22, 33, 34, 61], each of the slices are attached to hardware structures known as a C-Box or CBo, one per CPU core [19]. From experimentation with eviction sets [52] on Intel processors with inclusive LLCs, the set mappings of the eviction addresses imply the existence of double the number of slices than reported by the processor.[1] This can be characterised as each slice being split into two sub-slices per CBo, as the dashed lines in Figure 1 illustrate.

Linear Functions. When the number of LLC slices is a power of two, the processor calculates the slice index bits using a straightforward linear XOR operation on the physical address bits. This technique employs a set of processor model-specific permutation masks to select appropriate address bits and determine the slice index [33, 34].

Non-Linear Functions. When the number of LLC slices is not a power of two, the employed function is not linear. Instead, it consists of two phases, a linear XOR permutation selection similar to the linear functions, piped into a secondary stage which outputs slice values in the appropriate range. Yarom et al. [61] describe this secondary step, which can either be represented as a logic circuit or a lookup of mappings for the slice values termed a *base sequence* by McCalpin [34]. The base sequence is a sequence of slice values which are indexed by the output of the linear XOR operation. In this work we treat the secondary function as such a sequence.

1. As reported by reading Model Specific Register `0x396`.

2.3. LLC Contention-Based Cache Attacks

The last-level cache (LLC) has become a compelling target for contention-based attacks as it is shared across all CPU cores. It permits information leakage between cores of the same physical processor, with several prior attacks exploring this possibility [5, 32, 42, 58, 62, 64]. In these cross-core attacks, an attacker first primes the shared cache with their own memory using *eviction sets* to move the victim's data into RAM. The attacker then waits for the victim to run and displace the eviction set memory. Through, e.g. observation of the access times to their own data, an attacker can infer the victim's memory accesses, allowing for the recovery of sensitive information such as cryptographic secret keys [8, 21, 32, 62] and user interface interactions [36, 46].

2.3.1. Attack Workflow. The main goal of the attacker is to leak and recover secret data from other CPU cores. To do this, the attacker needs to build many minimal eviction sets [47, 52], i.e. the smallest set of *congruent* addresses which evict a target address into the next cache level or to RAM. Two addresses are described as *congruent* when they both map to the same cache set as each other. Given an n-way set associative cache, a minimal eviction set contains exactly n addresses. After this, the attacker needs to narrow down which eviction sets correspond with the victim's memory to observe their behaviour [1, 11, 32, 64].

For an inclusive LLC, the attacker can evict the victim's memory out of the LLC (and hence the private caches) to observe memory access patterns [32, 52]. However, for non-inclusive LLCs, the coherency directory (CD) or snoop filter (SF), as determined by the processor family or configuration settings [17, 20, 53], provides an accessible structure to cause private cache evictions [58, 64]. These hardware structures are a finite resource tracking the locations of cached data in the LLC. Thus, a congruent set of addresses which map to the same CD or SF set can likewise cause evictions into RAM.

2.3.2. Finding Eviction Sets. One of the main challenges in cache attacks is finding minimal eviction sets in a virtual memory environment. With virtually addressed 4 KB memory pages, the attacker does not know the higher order bits of the physical address which decide the cache set index for the L2 and LLC. Moreover, these bits are also required for the computation of the slice index with the hash function. Therefore, the attacker cannot directly create eviction sets for these caches. To find a minimal eviction set, the attacker instead needs to find an eviction set by testing for contention. The attacker initialises a large candidate address pool which evicts the target address from the desired cache and then reduces this pool into a minimal eviction

set with a chosen pruning algorithm [28, 42, 47, 52, 64]. These algorithms require test functions to determine whether a candidate set can actually evict a target address, with differing techniques for inclusive caches [32] and non-inclusive caches [58].

Group Testing. Vila et al. [52] describe the group testing method for finding minimal eviction sets for the LLC. Here, the attacker splits the candidate set into $n+1$ groups (with n being the associativity of the cache) and tests the first n groups together for eviction of the target address. If the first n groups successfully evict the address, then the last group can be pruned away, with the process repeated on now the smaller candidate set. If the target address is not evicted, then the attacker splits away a different group and tries again.

Prime+Scope. The Prime+Scope eviction set construction method for the LLC [42] works by first accessing the target address, then repeatedly guessing and accessing further addresses which may be congruent. After each guess, the attacker checks if the target address had a fast access or not. If it was fast, then the guessed address is appended to the eviction set. This entire process repeats until a minimal eviction set of size n is built.

Binary Search. In binary search pruning [64], the attacker incrementally expands a growing window of addresses to test, checking them in sequence. The window is expanded until it contains enough congruent addresses to evict the target. Once an eviction is confirmed, the last address in the window is moved to the front, and the attacker continues searching. When the first n addresses are confirmed to be congruent, a minimal eviction set has been identified.

Prune+PlumTree. The Prune+PlumTree algorithm aims to find many independent eviction sets simultaneously from an initial candidate set [28]. An initial pruning step identifies cached memory by accessing the entire candidate set, discarding memory which experiences a cache miss. In the second stage, called PlumTree, the pruned candidates are recursively organised into minimal eviction sets by imposing a tree structure on the candidates. This partitioning allows the algorithm to identify possible minimal eviction sets at the leaf nodes, discarding those at leaves which cannot evict other memory.

2.3.3. Optimisation Techniques. More advanced techniques can improve the efficiency of eviction set generation, one such example being conflict set reduction [32, 52]. This conflict set is the union of all minimal eviction sets for the LLC, and is then split into individual eviction sets. It acts to filter the initially large candidate set by reducing the number of addresses to test, speeding up the overall process.

Another improvement is the concept of L2 candidate set filtering [64]. This technique works by reducing the number of addresses to test by filtering out those which do not correspond to the same L2 set as the target address using an L2 eviction set. In the same work, the authors also propose using a parallel access method to reduce the time taken to test the eviction sets, reducing false positives when testing for eviction.

Previous works [32, 61] exploit a characteristic of linear slice functions to *propagate* eviction sets from one offset to all offsets within a page. On machines with this type of hash function, adjusting the offset bits of each address in a single eviction set consistently maps them to the same LLC slice and set across all page offsets.

2.4. Weird Gates

To reduce stalls and delays, modern processors may execute instructions out of order. For that, the processor tracks the data dependencies of instructions and aims to execute instructions as soon as their dependencies are satisfied, even if preceding older instructions have not completed execution. To increase the benefit of out-of-order execution, processors try to predict the outcome of control-flow instructions, such as branches, and execute instructions along the predicted path even before the processor determines the outcome of the branch. In the case of correct prediction, this allows the processor to progress execution beyond the branch. Conversely, if the process eventually determines that the prediction was incorrect, the processor squashes the mispredicted execution path and resumes execution from the correct branch outcome.

Squashing speculative execution does not undo the effects that the squashed instructions have on the microarchitectural state of the processor. Weird gates [24, 26] exploit this observation to compute various functions on microarchitectural state. Specifically, a typical weird gate consists of two or more *instruction chains* [14], possibly non-consecutive sequences of instructions, such that each instruction in the chain has a data dependency on earlier instructions in the chain. The last instruction in a *control chain* is a mispredicted control-flow instruction, whose outcome depends on executing all of the instructions in the chain. Examples of such mispredicted control-flow instruction include conditional branches [26] and return instructions [24].

Another type of chain is a *signal chain*. Such a chain contains one or more "signal" instructions along the mispredicted path, whose execution leaves observable traces in the microarchitectural state of the processors. A common example of such signal instructions is memory accesses that, when executed, result in bringing the accessed memory to the cache.

This design of weird gates creates an observable race between the signal and the control chains. Specifically, when the execution reaches the end of the signal chain, the processor determines that a misprediction has occurred and terminates the speculative execution of the signal chain. If the speculative execution reaches a signal instruction before it is squashed, the instruction will leave its trace in the microarchitecture. Conversely, if the speculative execution is squashed before executing the signal instruction, the instruction will not be executed, leaving no trace.

Weird NOT Gate. As a concrete example, we now describe a weird NOT gate [24, 26]. The gate is implemented as a function that takes two memory addresses, `input` and

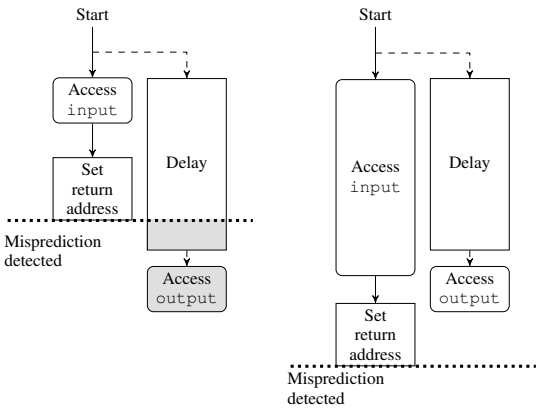

Figure 2: Operation of the weird NOT gate, which inverts the cached state of `input` to `output`. (Adapted from: Horowitz et al. [14].)

`output`. We assume that on invocation the `output` address is not cached, but make no assumption on the caching state of the `input` address. The aim of the gate is that after its execution, `output` will be in the cache if and only if originally `input` was not in the cache.

Figure 2 illustrates the operation of the gate in the case that `input` is in the cached (left) and when it is not (right). The signal chain consists of a delay, generated by a sequence of arithmetic instructions, followed by an access to `output`. The control chain consists of an access to `input`, followed by updating the return address so that executing the return instruction skips the execution of the signal chain.

When `input` is in the cache (Figure 2 left), retrieving its value is fast and the control chain completes before the control chain accesses `output`. Hence, in this case, `output` remains uncached. Conversely, if `input` is not in the cache, accessing it will be longer, delaying the detection that the return is mispredicted. This would allow the signal chain to access `output`, bringing its contents to the cache.

3. Overview

The primary goal of our work is to increase the feasibility of contention-based cache attacks by speeding up LLC eviction set generation, the initial step in contention-based cross-core cache attacks. The main challenge we overcome is the uncertainty posed by the LLC slice function. We achieve that through our slice prediction technique in Section 4, where we group memory by slice based on observed microarchitectural effects using a weird gate for memory access time comparison.

We further enhance our slice-mapping approach in Section 5 by propagating fewer slice guesses across entire 4 KB memory pages using knowledge of the processor's slice function, accelerating the memory classification speed while additionally reducing errors. This allows us to infer nearby address slices without needing to measure them directly by finding the closest matching whole-page slice permutation.

Finally, we bring these efforts together to improve LLC eviction set generation with slice-aware optimisations in Section 6. We combine our approach with state-of-the-art L2 candidate set filtering by Zhao et al. [64], comparing to their work as well as the Prune+PlumTree algorithm [28].

Threat Model. Our work follows the standard cross-core attack threat model where the adversary can only execute *unprivileged* code on the same physical processor as the victim, but not on the same physical core. We assume a sliced last-level cache, as used in modern Intel processors, with a known slice function (having recovered it offline, a one-time reverse engineering effort). We do not require the use of huge pages, relying solely on standard 4 KB memory pages (i.e. we do not have visibility of the higher-order bits for the addresses of memory we use). Moreover, we do not disable any of the hardware prefetchers.

4. Determining Slice Mappings for Memory

In this section, we demonstrate our methodology for determining LLC slice indices for memory without requiring root access. Our eviction set generation routine leverages the ability to group memory addresses based on their slice mappings to gain several speed improvements by exploiting observable differences in access latency to the LLC. We experiment with several options to predict slice mappings for memory, exploring the use of the RDTSCP instruction as well as a variant of a weird NOT gate to measure LLC access times. However, we find that under high system noise both approaches fail to accurately predict the LLC slice of memory addresses.

We then introduce the *comparator* gate, a new type of weird gate that compares access timing of two memory addresses, instead of using a fixed delay, as in prior weird gates designs. We find that this comparison approach is less sensitive to noise than the other methods and that it consistently maintains high classification accuracy, regardless of which core we measure from.

L2 Eviction Sets. Throughout this section, we require the use of L2 eviction sets to place memory addresses in the LLC to then measure access times using several techniques. An L2 eviction set consists of a set of memory addresses that, when accessed, evict a target address from the private caches into the shared LLC, enabling our experiments. Generating L2 eviction set only takes tens of milliseconds (depending on the processor) due to the small size in comparison to the LLC and the lack of slice function. We use group testing [52] for L2 eviction set creation and use small 4 KB memory pages when creating to mimic the scenario of a real-world system.

4.1. Predicting Slices with Access Time

The sliced LLC implemented by Intel exhibits different access times depending on the slice a memory address belongs to [6]. This has been explored in previous works as a method to reverse-engineer the slice function [61], as

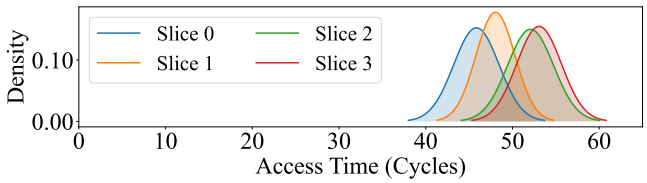

Figure 3: LLC slice access latency probability distributions for the four slice i7-6700K, measured from core zero.

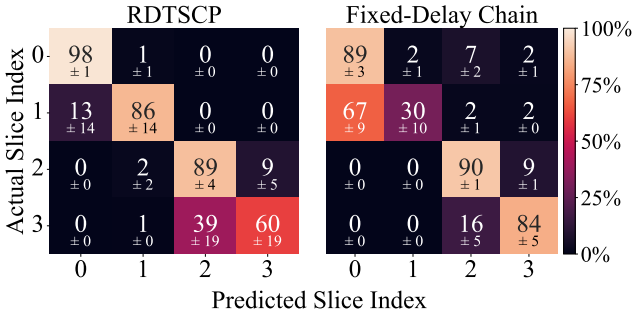

Figure 4: RDTSCP and fixed-delay chain slice classification accuracy in a low-noise system, measured from core zero.

well as to create covert communication side-channels and mount cross-core attacks [4, 39]. As visualised in Figure 1, cores with memory residing in distant slices must wait for their traffic to travel further than if the memory resides in a closer slice.

4.1.1. LLC Slice Access Time Variance. To start our investigation, we first validate that LLC slices indeed exhibit different access times. We execute the experiment on an Intel i7-6700K processor, which has four physical cores and four slices. We first create L2 eviction sets, which allow us to evict the tested memory lines from the L2 cache. We then set the process affinity to core 0 and use the RDTSC instruction to measure access latency from that core to the memory lines.

To determine the slice each memory line maps to, we use the `/proc/<pid>/pagemap` interface. We then use the published LLC slice function for the i7-6700K [33] to determine ground-truth slice mappings and check the accuracy of our predictions. In Section 5.1 we show how to determine the slice in an unprivileged scenario, i.e. without access to `/proc/<pid>/pagemap`.

We average 10,000 accesses to each slice over 100 independent executions to establish their access time distributions. We take 100 overall samples for this experiment as we observed slice timings to differ between runs due to frequency changes in the core and uncore. This gives an indication of the global access time distribution for each slice. We repeat individual address measurement when the reported cycles are greater than 100, indicating a RAM access due to spurious microarchitectural noise (e.g. cache eviction due to system interrupts).

Figure 3 shows the distribution of access times to each slice. We note that the access times distributions differ between slices, although they overlap. This implies that information from access times can be used to predict the slice index of a memory address.

4.1.2. RDTSCP Predictions. We now reverse the scenario. Instead of trying to find the time to access a slice, we attempt to determine the slice by measuring the access time. To calibrate our experiment, we first compute the average access time to each slice, averaging over 1,000 measurement for each slice. We then measure the access time to 10,000 memory addresses, and compare the access time to the calibrated measurements to predict the slice each memory address maps to. Due to the overlap observed in Figure 3, a single timing measurement is likely to be a bad predictor. To reduce the noise, we average the result of 10 measurements and use the average to predict the slice. As before, we ignore outliers with a value above 100 cycles.

The confusion matrix in the left side of Figure 4 shows the slice prediction accuracy and the standard deviations. We achieve an average prediction true positive accuracy of 83% by normalising the confusion matrix to percentages. While averaging timing samples enhances accuracy, there is still room for improvement, as it is not optimal for all slices. Notably, this method cannot easily differentiate between slices two and three due to their overlapping access times. Although the attacker could use more advanced timing analysis techniques to improve accuracy [3, 25, 51], these require in-depth analysis of the noise distribution.

4.1.3. Weird Gates for Access Timing. We now propose a different access time measurement technique using recent weird gate concepts. Recall that such gates use microarchitectural races to enact certain behaviour such as logical operations, re-purposing the memory hierarchy to perform computation. We seek to exploit this behaviour to instead measure slice access times with higher resolution than RDTSCP.

Our core idea is that the NOT gate of Kaplan [24], illustrated in Figure 2, already compares the execution time of a memory access to that of a fixed delay. To that aim, we place `input` in the LLC, but evict it from the L1 and the L2 caches. Our aim is to rely on the difference in access time to various slices in order to determine in which LLC slice `input` is cached. For that, we try to set the length of the delay in the gate such that the access to `output` loses the race if `input` is cached in a closer LLC slice, but wins it if `input` is in a further slice.

We construct the delay from a chain of dependent `add` instruction, which we assume take one cycle each. Increasing the number of `add` instructions increases the delay before the access to `output`. We then test the gate with different `input` addresses, each residing in a different LLC slice. Figure 5 shows the probabilities of the delay chain winning as functions of the length of the chain. We observe

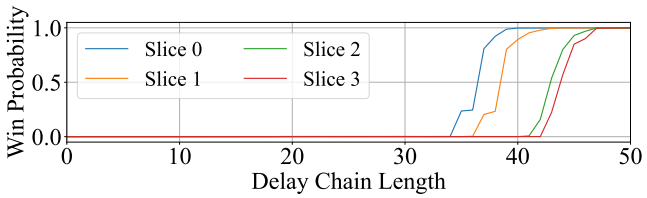

Figure 5: Fixed-delay chain gate win probability as a function of the delay chain length, measured from core zero.

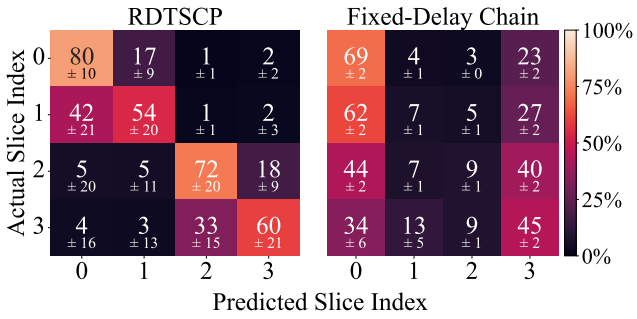

Figure 6: RDTSCP and fixed-delay chain slice classification accuracy in a busy system, measured from core zero.

a clear distinction in tipping points for each slice, indicating that the gate can distinguish each of the four slices. As a result, we can now try to use the delay chain length as a unit of measurement for the memory's LLC access time.

4.1.4. Fixed-Delay Chain Predictions. Having found that different slices show tipping points at different delay lengths, we now design an experiment to assess the possibility of using modified NOT gates to predict the slice a memory location maps to. The core idea is to measure the tipping point for a memory location and compare it to the known tipping points for each slice. In more details, we first calibrate the experiment, determining the tipping point for each slice. For that, we select a memory location for which we know the slice, and search for the delay length at which the win probability of the gate changes. We repeat the process for each slice, finding the tipping point for each.

We then attempt to predict the slices of 10,000 memory addresses. For each address, we search for the tipping point. Once found, we compare to the tipping points obtained during the calibration and select the closest one as a prediction for the slice. Figure 4 (right) shows the prediction success. Although this method can distinguish between slices two and three better than RDTSCP, its average true positive accuracy is not as high as overall at 73%. In particular, it appears that this method fails to accurately distinguish between slices one and two.

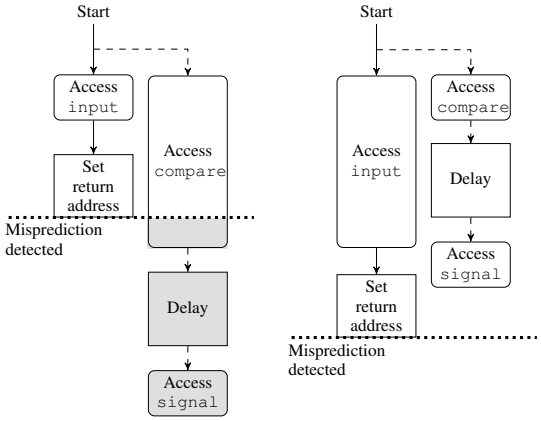

Figure 7: Operation of the comparator weird gate. Left when `input`'s latency is less than `compare`, right when it is not. Shaded instructions never execute.

4.2. Slice Prediction Under Noise

The experiments in Section 4.1 were undertaken under relatively quite system conditions, with no significant system activity. To evaluate slice prediction in more realistic scenarios, we now repeat the experiments in the presence of activity. We employ *stress-ng* [29] to generate uncore traffic with a cache stress preset running on all non-measuring cores. All elements of either experiment remain the same, except for the increase in system activity.

Figure 6 summarises the results. For RDTSCP, the prediction accuracy now drops from the average of 83% in the quiet scenario to 67% in the noisy scenario. The granularity of this timer is too coarse, with overlapping predictions for slices 0–1 and 2–3. The variance in access times likewise increases, causing prediction errors. The prediction accuracy of the weird gate method drops even further, i.e. from 73% in the quiet scenario to only 32% in the noisy scenario, as shown in the right side of Figure 6, indicating that this measurement technique is overly sensitive to the processor's frequency and noise. In conclusion, although the techniques we investigate in Section 4.1 have an acceptable accuracy when the system is idle, their accuracy drops significantly when the system is busy.

4.3. The Comparator Weird Gate

We now present the comparator gate, a new weird gate design that allows predicting the LLC slice of a memory address even in the presence of system noise. We observe the underlying cause of the noise is that the access latency to slices is not fixed but depends on environmental factors, such as fluctuating core frequencies and varying uncore load. In contrast, both approaches presented in Section 4.1 do not account well to these factors. The RDTSCP approach measures latency against the wall clock, which has a fixed frequency, whereas the weird gate measures latency against the core clock, which is independent of the uncore clock

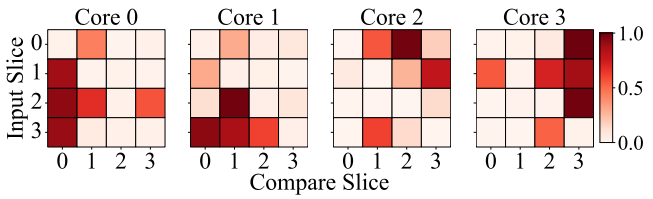

Figure 8: Average comparator gate `signal` access probability when compared across each slice and core. Darker indicates the signal memory was accessed, and hence `compare` slice access was faster.

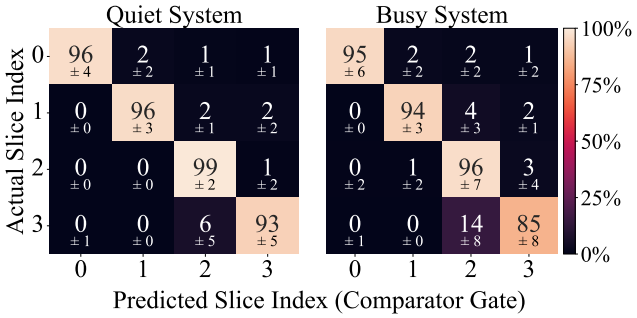

Figure 9: Comparator gate slice classification accuracy, measured from core zero.

and load. Because variations due to environmental factors are bigger than the differences between access latencies to different LLC slices, measurements that do not account for environmental factors are doomed to fail.

To overcome this challenge, we propose to avoid independent clocks and instead compare the access time to a memory address against the instantaneous access latency to the LLC slices. For that purpose, we design the *comparator* weird gate, demonstrated in Figure 7. Unlike prior weird gates [24, 26, 54], which compare a memory access to a fixed-delay chain, our comparator gate races two independent memory accesses. Specifically, our comparator gate creates a microarchitectural race between the access times to two memory addresses: an `input` address, which we wish to determine the slice it maps to, and a `compare` address, which is in a known slice. We use a `signal` to indicate whether the access to `compare` takes longer than the access to `input`. To compensate for the time required for setting up the return address, we need to add a small delay after accessing `compare`. We experimentally determine the length of this delay to balance the lengths of the chains when accessing addresses in the same slice. We provide the implementation of the gate in Appendix A.

To determine whether the comparator gate can distinguish between different slices, we test all combinations of `input` and `compare` slices. For the experiment, we use 10,000 addresses, taking 10 measurements for each address from each core. Figure 8 shows the average signal value. As the figure demonstrates, the patterns of the signal values for each `input` slice depend on the core the experiment runs on. However, on each core different `input` slices show different signal patterns. Thus, we can use signal patterns as predictor for the slice.

4.4. Comparator Gate Evaluation

We now evaluate whether the comparator gate is suitable for slice predictions. For that, we use a set of compare addresses, one for each slice. In this section, we rely on known ground-truth for generating this set. Section 5.1 shows how to generate the compare set without elevated privileges.

As Figure 8 shows, an input address does not always win or always lose the race against a given compare address. Instead, there is a win probability that depends on the slices of both the input and the compare addresses. Consequently, to determine the slice an input address maps to, we need to execute the comparator gate multiple times for each pair of addresses to determine the win probability.

Our method for determining the slice a memory address maps to consists of two main steps. In the profiling step, we measure the win probability for inputs of known slices against the memory addresses in the compare set. The profile consists of a set of vectors, one for each input slice. The vector includes one coordinate for each compare slice, whose value is the win probability for an input of the given input slice against the compare address in the compare slice. To ensure the accuracy of the profile, we use 1,000 invocations of the comparator gate to determine each vector coordinate. Figure 8 can be viewed as visualising four such profiles, one for each core. Each vector in the profile is visualised as a horizontal line matching the input slice.

Once the profile is created, we move to the second step of predicting the slice of a memory address. For that, we use the comparator gate to test the input memory address against each of the addresses in the compare set. We repeat the measurement 10 times for each compare slice, creating a probability vector that indicates how likely is the input slice to win against each of the elements in the compare set. We then compute the Euclidean distance between this vector and each of the vectors in the profile and use the input slice of the closest vector as the predicted slice index.

Figure 9 displays the resulting slice prediction confusion matrix. It shows that our method does not suffer from the same noise-related accuracy loss as RDTSCP and the fixed-delay chain gate. This gate demonstrates 96% and 93% overall prediction accuracy in the quiet and the busy scenarios respectively.

Multi-core. As Figure 8 shows, the profile depends on the core on which we execute the experiment. To evaluate our slice prediction method with the different profiles, we repeat the experiment running on each processor core. Table 2 summarises the results, showing the true positive classification rates for each slice. Our method maintains a 97% true positive rate when averaged across all cores and

TABLE 2: True positive comparator gate classification percentage rates for each slice and core on a quiet system.

Core	Slice 0	Slice 1	Slice 2	Slice 3	Average
0	96 ±4	96 ±3	99 ±2	93 ±5	96 ±4
1	99 ±1	95 ±5	95 ±2	99 ±5	97 ±3
2	98 ±6	95 ±3	94 ±6	99 ±1	97 ±4
3	93 ±7	98 ±4	95 ±8	96 ±8	96 ±7

slices, showing that it can adapt to any processor affinity.

Profile Stability. The attacker is unlikely to have an a-priory knowledge of the core they execute on. Moreover, we find that the profile may vary between executions, even on the same core. To allow executing the attack without knowledge of the core and to overcome profile variations, we compute the profile before each execution of our eviction-set construction algorithm.

5. Intra-Page Slice Mappings Propagation

In Section 4, we discuss our method for classifying memory by slice. This requires measuring every address in a page, which is time-consuming while having a non-zero error rate. In this section, we leverage knowledge of the slice function's structure to assist in two key areas. We first introduce our technique for determining our compare set addresses for the comparator gate to run our profiling step. Then, we describe how we can propagate slice classifications within a memory page to reduce the number of measurements required.

We observe that for each virtual memory page, the function implies a finite number of mappings between page offsets and LLC slices, which we refer to as a page-slice mapping. These mappings result from the dependency of the slice function on both the known lower physical address bits (the offset in the page) and the unknown upper address bits. Our goal is to determine the correct page-slice mapping for a given page. Our first method, *closest match*, involves using our comparator gate to map all offsets in a page to their corresponding slices and selecting the mapping which matches the most measured slice indices. The next, *Bayesian inference*, determines the most likely page mapping based on a dynamic probability model measuring only a subset of offsets. The final method, *decision tree*, consists of a guided search based on maximising information gain to find the page-slice mapping with the fewest measurements. We evaluate these methods on several Intel processors, determining the overall execution speed and resulting accuracy in comparison to the default comparator gate method.

Structure of the Slice Function. Recall that the LLC slice function is a proprietary function used by Intel processors to determine the slice index for a given physical address. The main property we want to exploit is that the slice function generates mappings of physical memory cache line offsets to slice indices.

Several prior works detail the observed structure of the proprietary function which uses either a linear or non-linear approach to calculate the slice index depending on the number of slices and the processor [10, 15, 16, 22, 33, 34, 61]. For linear functions, the slice index is calculated by an XOR operation on the physical address bits using several permutation selection masks. This equally distributes cache lines across the slices. In contrast, non-linear functions have a two-phase approach, starting with a linear XOR permutation selection, piped into a secondary stage which outputs the slice index in a range based on the XOR result.

We observe the number of unique page-slice mappings is equal to the number of slices for linear functions. For a CPU with n slices, this results in mapping consecutive cache line offsets in a page to the n different slices. This is due to the equal distribution design, through interactions between the XOR operation involving both known lower address bits and unknown upper bits. For example, an i7-6700K generates the following four mappings (where mappings B, C, and D, are simply A XORed with 0x1, 0x2, and 0x3 respectively):

Mapping. A: `0123 0123 ... 2301 2301`
Mapping. B: `1032 1032 ... 3210 3210`
Mapping. C: `2301 2301 ... 0123 0123`
Mapping. D: `3210 3210 ... 1032 1032`

For non-linear functions, the number of unique page-slice mappings is greater than the number of slices and depends on the processor. Processors with six and ten LLC slices have 128 unique page-slice mappings.

We detail our method for reverse-engineering the slice function in Appendix B, and provide this as an auxiliary tool in our open-source repository.[2]

5.1. Determining the Compare Set

Our unprivileged threat model restricts us to 4 KB memory pages with access to only the lower 12 bits of the physical address. Consequently, we do not know the ground-truth slice mappings. To run the profiling step for the comparator gate, we must first determine the compare set of addresses, one in each slice, despite not knowing the ground truth. Therefore, we need to determine the slice mappings for a single virtual page of memory to find our compare set and also addresses with known slices for calibration purposes.

Processors with linear slice functions with n slices generate n unique page-slice mappings, where we can choose n pre-determined offsets for any page which are guaranteed to map to n different slices. The rest of the offsets are used as calibration targets to initialise the comparator gate vectors. We note that we only know that the addresses in the compare set are in different slices, but we do not know the exact mapping (i.e. which address maps to which).

We cannot use this approach for processors with non-linear functions because we cannot assume that pre-determined n offsets in a page will map to n different slices. In order to find the compare set, we try to identify the slice mapping for the page we are targeting. Once we identify

2. https://github.com/0xADE1A1DE/Slice-Slice-Baby

this, we can select n offsets that map to the n slices. We use our comparator gate to run pairwise comparisons for all offsets in the page. This creates a comparison pattern based on closer versus further away slices. We then count how many times the measurement pattern from the gate matches each of the page-slice mappings. We select the mapping with the highest number of matches. It is important to note that this approach does not fully identify the slices, as several equivalent mappings can appear from our measurements. However, this does not prevent the technique from working, and the profile we generate can successfully distinguish different slices and partition the memory without knowing the ground truth. For simplicity, the rest of the discussion assumes that the determined function is correct, rather than an equivalent function.

5.2. Closest Match

The first approach we explore as a means to increase the classification speed while maintaining accuracy for the slice mappings predictions is the closest match. This method involves comparing all addresses in a page to the compare set using the comparator gate, then selecting the mapping which matches the majority of predicted slice indices. The method works for both linear and non-linear slice functions. It results in much higher accuracy than the default comparator gate method, as it uses knowledge of the slice function to make predictions. However, as it still requires comparing each address to the comparison set, so it does not improve the classification speed.

5.3. Bayesian Inference

Our next approach involves the use of statistical Bayesian inference to guess the page-slice mapping. The aim here is to maintain probabilities for each page-slice mapping, and update such probabilities based on iteratively measured slice indices to determine the most likely mapping. At the beginning, each page starts with an equal probability of occurrence as we have no prior information. We then predict the slice index for the first offset in the page using the comparator gate. We update the posterior probabilities of each unique page mapping given the predicted slice index and move to the next offset to repeat. When we reach a given threshold (0.90), we select the mapping with the highest probability as the page's slice mapping and can now proceed to the next page. This results in a fewer number of offsets measured per page, which we discuss further in Section 5.5.

5.4. Decision Tree

The decision tree method represents a guided search of slice indices through specific page offsets to determine the overall page-slice mapping.[3] We illustrate this structure in Figure 10. Its design maximises the information gained at each decision node, reducing the number of overall slice predictions.

3. This method is equivalent to the ID3 algorithm by Quinlan [44].

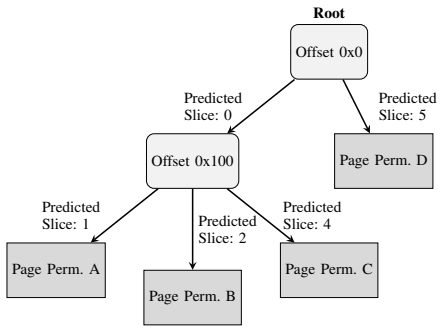

Figure 10: Example decision tree structure. Nodes represent page offsets to predict the slice for. Leaves represent the determined page-slice mapping.

Building the Tree. Generating the decision tree involves a recursive construction of each decision branch as we narrow down to each unique page-slice mapping. To construct the first node, we select the page offset which provides the highest distinguishing entropy across all possible mappings. Note that for linear mapping functions, all offsets have the same distinguishing entropy at the beginning. However, this is not necessarily the case for non-linear mappings.

Specifically, to calculate the entropy for an offset o, we count the occurrence of every slice index at the offset for each of the n mappings P:

$$\forall i \in \{0, 1 \ldots, n-1\}, \quad v = P_{i,o}$$
$$slice_counts_v \mathrel{+}= 1$$

Then by using Shannon's formula, the offset's uncertainty H is calculated by:

$$H = -\sum_{s=0}^{LLC_SLICES-1} p(s) \log_2 p(s)$$

where $p(s)$ is the probability of the slice index occurring:

$$p(s) = \frac{slice_counts_s}{n}$$

From this, we can select the offset with the highest entropy. We then use our gate to compare this offset to our comparison set. Then, we create new child nodes for each possible slice index, forming the edges of the tree, and remove the offset from the list of potential measurements. With each child node, we repeat the above process using the subset of page-slice mappings which matches the slice index associated with that node. The recursion continues until only one mapping remains. At this point, we create a leaf node to store the unique mapping, which we use to determine the slice function for that page.

The user can select multiple offsets to measure at each decision node, which can improve the accuracy of the method at the detriment of speed. In practice, we find that the number of offsets to measure per decision node depends on the processor and the slice function. We provide configurations for several processors in our codebase.

TABLE 3: Comparison of intra-page propagation methods for slice predictions. True positive classification accuracy listed as a percentage, speed in MB/s with respective standard deviations. Bold values indicate the best performance.

		Comparator Gate		Closest Match		Bayesian Inference		Decision Tree	
Processor	**Linear**	Accuracy	MB/s	Accuracy	MB/s	Accuracy	MB/s	Accuracy	MB/s
i7-6700K	✓	84 ±4	70 ±1	100 ±1	70 ±1	**94 ±4**	**568 ±46**	90 ±7	538 ±38
i7-11700KF	✓	77 ±11	27 ±0	97 ±4	27 ±0	**79 ±10**	**323 ±20**	71 ±11	352 ±23
i7-13700H	✓	58 ±9	21 ±1	96 ±3	21 ±0	**74 ±10**	**175 ±31**	52 ±9	198 ±54
i7-8700	✗	79 ±11	45 ±6	98 ±7	48 ±1	98 ±7	70 ±4	**91 ±9**	**119 ±32**
i7-9850H	✗	79 ±9	51 ±1	68 ±10	50 ±0	99 ±6	72 ±4	**95 ±5**	**187 ±31**
i9-10900K	✗	74 ±9	32 ±6	99 ±4	34 ±0	99 ±5	60 ±3	**96 ±3**	**96 ±21**
i9-12900KF	✗	53 ±9	25 ±0	95 ±4	25 ±0	**93 ±4**	**41 ±2**	74 ±6	19 ±4
i9-14900K	✗	52 ±12	15 ±0	96 ±6	15 ±0	**95 ±7**	**19 ±1**	82 ±8	6 ±2

Usage. Starting from the root node, we compare the indicated offset to the comparison set and predict the slice. We then traverse to the child node corresponding to the predicted slice and repeat until we reach a leaf node with a page-slice mapping. If a child for the predicted slice does not exist, this constitutes a slice prediction error. In this case, we traverse up to the parent node and retry. However, this may still cause the wrong mapping to be selected if an uncaught error occurs higher up in the tree. We leave this for future improvement.

5.5. Evaluation of Propagation Methods

Using the same methodology as in Section 4, we evaluate the propagation methods for their accuracy and classification speed. We first place our desired memory into the LLC using L2 eviction sets, and use the propagation methods to determine the slice mappings for each page. For each method, we measure 10,000 cache lines across the LLC, and average the results over 100 runs to get a global view of their performance. As the decision tree can be built statically, we do not record the time taken to generate the tree in these measurements.

We compare the classification speed and accuracy of each method across several Intel processors, summarising the results in Table 3. The best improvement in accuracy comes from the closest match method, which is to be expected considering that the method combines the most measurements with the knowledge of the slice function.

For linear slice functions, the dynamic Bayesian inference method is the fastest and most accurate due to the nature of the hash function. On our i7-6700K processor, we find this method requires measuring between two and three addresses per page on average to determine the page mapping with confidence. For the i7-11700K and i7-13700H processors, this increases to five offsets per page on average.

The decision tree method significantly improves the classification speed for most processors while maintaining a good level of accuracy, working especially well for non-linear functions. We reach above 90% accuracy for these processors. On the i9-12900KF and i9-14900K, the decision tree method performs relatively poorly due to poor baseline comparator gate performance. For such processors, we choose the Bayesian inference method, which achieves higher speed and accuracy because it dynamically adjusts the number of measurements required when trying to determine the page-slice mapping.

6. Slice-Aware Eviction Set Generation

To successfully undertake a contention-based cross-core attack, the attacker must first generate eviction sets for every LLC set. In this section, we introduce three optimisations for this procedure, leveraging our foreknowledge of memory slice mappings.

The first, *candidate set slice filtering*, reduces initial eviction candidate sets to only contain memory with the same slice as the target address. The second, *non-linear eviction set propagation* unlocks the ability to mirror eviction sets to other page offsets on processors with non-linear slice functions. The third, *test eviction filtering* accelerates the search for a target address for which we need to build a new eviction set. We demonstrate a significant improvement in execution time for processors with both linear and non-linear slice functions, building on state-of-the-art eviction set generation techniques e.g. L2 candidate set filtering and parallel probing [64] and outperforming Prune+PlumTree [28].

6.1. Implementation Details

Before describing our optimisations, we first detail some techniques and configurations that we use in the implementation of our eviction set creation methodology.

Test Eviction Function. Current eviction set creation methods use a *test eviction* function to determine whether the given set of addresses can evict a target address from the LLC. Similar to Zhao et al. [64], we place all eviction set addresses in an array. This lowers the execution time by exploiting memory-level parallelism, making this method more efficient and noise-resilient in comparison to a linked list approach [42, 52]. We access the elements in the array with a user-configurable traversal pattern to suit the processor's replacement policy. We also access another virtual address in the same page prior to measurement to ensure caching of the TLB entry, reducing false-positives for eviction [8].

Choice of Pruning Algorithm. We use the optimised group testing pruning algorithm from Zhao et al. [64]. We find

this version of the pruning algorithm to be more efficient than the original using early termination [52]. The original group testing algorithm uses an early termination approach, identifying and pruning removable groups as soon as they are found, without needing to search through all remaining groups. We also find that the optimised version performs better than the binary search algorithm [64].

L2 Candidate Set Filtering. We integrate the L2 candidate set filtering technique [64] into our eviction set generation process. The approach builds on the observation that if two addresses map to the same LLC set, they also map to the same L2 set. Consequently, filtering by L2 set reduces the number of addresses for the pruning algorithm to check for congruency with the target address.

6.2. Our Optimisations

Building on our insights from Section 4 and Section 5, we implement three specific optimisations to speed up the generation of eviction sets for the entire LLC.

Candidate Set Slice Filtering. We carry out slice filtering for our candidate set addresses to minimise the execution time of the pruning algorithm, similar to L2 candidate set filtering. We use our knowledge of predicted slice mappings to filter the initial candidate set to consist of addresses that map to the same slice as the target address. This increases the likelihood of finding congruent addresses, dividing the total size of the candidate set by the number of LLC slices. The speedup depends on the algorithmic complexity of the chosen pruning method and the number of slices.

Eviction Set Propagation for Non-Linear Slice Functions. Linear slice functions enable straightforward propagation from a single page offset to the rest of the page [32]. For example, constructing a single eviction set starting at page offset `0x0` consists of addresses which are congruent with one another, mapping to the same LLC sub-slice and set. If we add a fixed page offset to each of the addresses, the new *mirrored* eviction set addresses will all map to the same LLC sub-slice. This technique can be applied to any offset, meaning we only need to find eviction sets for a single page offset with linear functions.

Propagating eviction sets for non-linear slice functions in the same manner results in a faulty eviction set with addresses mapping several slices [61], being unable to cause cache contention. To solve this problem, we first generate a single eviction set, and then generate mirrors for the rest of the page offsets. We can use our previously obtained slice mappings (Section 5) to determine which addresses map to the same slice and check whether the mirrored eviction set maps entirely to a single slice. This does not require further contention tests. With this approach, we experimentally find we only need to generate on average 15% of all LLC eviction sets for processors with ten slices, and 22% for those with six slices using the conventional method.

Test Eviction Filtering. When generating multiple eviction sets across an entire system, we need to determine if any previously found eviction sets can evict the target address to avoid duplicates. For the i7-6700K, each single page offset has 128 potential LLC eviction sets. This number increases to 320 for the i9-10900K due to its larger LLC.

Past methods tend to test every previously-generated eviction set with the potential target addresses [47, 52]. We optimise this process by testing the subset of eviction sets that correspond to the same slice and L2 set as the target. This reduces the time between finding a target address and generating a new eviction set.

We can reduce the number of LLC eviction sets to test from 128 and 320 to just two in the aforementioned processors. They each have 16 L2 eviction sets per page offset, derived from 1024 L2 sets across 64 L1 sets. For the i7-6700K with four slices, we narrow the testing to just two eviction sets, given $\frac{128}{16\times4} = 2$. For the i9-10900K with ten slices, the same property holds with $\frac{320}{16\times10} = 2$.

TABLE 4: Comparison of different LLC cross-core attack frameworks and their prerequisites.

Cross-Core Attack	4 KB Pages	Userspace	Slice Aware	No Shared Memory
Irazoqui et al. [21]	✗	✗	✗	✗
Gruss et al. [11]	✓	✓	✗	✗
Liu et al. [32]	✗	✓	✗	✓
Yarom et al. [61]	✗	✗	✓	✓
Vila et al. [52]	✓	✓	✗	✓
Purnal et al. [42]	✓	✓	✗	✓
Didier and Maurice [4]	✓	✓	✗	✗
P+PT [28]	✓	✓	✗	✓
L2CS [64]	✓	✓	✗	✓
Ours	✓	✓	✓	✓

6.3. Evaluation and Discussion

We now evaluate our approach to generating eviction sets across a variety of Intel Core processors. Table 4 compares our approach to other recent works in the field of cross-core attacks. Using our slice-aware optimisations, we greatly reduce the execution time for the eviction set initialisation phase.

We perform eviction set generation across several Intel processors. We do not significantly alter the operating state of the processors, aside from differing capacity and speeds of memory. In each case, we initialise a candidate set buffer $3\times$ the size of the LLC to ensure an even comparison across each methodology. All hardware prefetchers are enabled. We include the time taken to generate L2 eviction sets.

We evaluate two test scenarios, the first *Page Offset* where we generate all possible LLC eviction sets only for a single page offset and the second *Full LLC*, where we generate eviction sets for the entire LLC. The *Page Offset* scenario allows for comparing between processors regardless of slice function, representing the performance of our method without propagation to any remaining page offsets. On the other hand, *Full LLC* demonstrates the increased feasibility our optimisations provide for the execution time of the entire LLC eviction set generation.

We compare our approach to two recent works, one using L2 candidate set filtering (L2CS) [64], and the other

TABLE 5: Comparison of prior eviction set generation methods, L2 candidate set filtering (L2CS) [64] and Prime+PruneTree (P+PT) [28] on several Intel processors, averaged over 1,000 independent runs. Bold values indicate the best performance.

		Page Offset (ms)			Full LLC (ms)		
Processor	**Cores**	**L2CS**	**P+PT**	**Slice+Slice Baby**	**L2CS**	**P+PT**	**Slice+Slice Baby**
i7-6700K	4	113 ±9	83 ±25	**77 ±7**	115 ±7	83 ±25	**79 ±9**
i7-11700KF	8	447 ±95	475 ±42	**301 ±85**	466 ±121	475 ±42	**320 ±102**
i7-8700	6	190 ±17	**173 ±71**	323 ±55	6683 ±443	11506 ±935	**812 ±230**
i7-9850H	6	183 ±21	**145 ±60**	255 ±62	6302 ±599	9616 ±734	**729 ±280**
i9-10900K	10	544 ±109	**300 ±106**	759 ±170	15485 ±846	19864 ±1073	**1569 ±354**

using the Prune+PlumTree (P+PT) algorithm [28]. To ensure a fair comparison for processors with linear slice functions, we always carry out eviction set propagation as this is a previously understood optimisation [28, 61].

We reimplement L2CS to cater for Intel Core processors as well as incorporate our slice prediction methodology. The public implementation of the P-PT technique [27] cannot work across multiple page offset. To counteract this, we instead build each offset individually, recording the execution time for just this portion of the program, summing them afterwards to permit comparison.

Results. Table 5 presents the execution times for the various eviction set generation techniques across multiple Intel processors.

The main benefits of using our slice-aware optimisations become clear when considering the *Full LLC* scenario for non-linear sliced processors. Here, we achieve a 10× speedup for the ten-slice i9-10900K, reducing the full LLC eviction set generation time from 15 and 20 seconds, with L2CS and P+PT, respectively, to 1.6 seconds on average. Likewise, we achieve 8× and 9× speedups for the i7-8700 and i7-9850H both with six slices. Our approach does not outperform the other methods for *Page Offset* due to the overhead of the slice classification routine. However, for such processors with non-linear slice functions, we must find eviction sets at all page offsets, nullifying the disadvantage [61].

Considering linear slice function processors, there is little difference between *Page Offset* and *Full LLC* for all works, as eviction set propagation takes a negligible amount of time, mostly hidden by noise. For the i7-6700K, our algorithm provides an improvement of 1.1× over P+PT and 1.5× over L2CS. For the i7-11700K, we achieve a 1.5× speedup over L2CS and P+PT.

Quality of Eviction Sets. To evaluate the quality of the eviction sets, we record the number of found eviction sets, occurrence of duplicates, and those which were missing. A duplicate maps to the same set and slice as another eviction set. A missing eviction set occurs when there were LLC sets which could not be evicted by any of the generated eviction sets.

In all runs, both our method and L2CS find at least 99% of the total eviction set count for the LLC. There was a negligible rate of duplicate and missing eviction sets, with ours marginally higher due to duplication of errors during eviction set propagation.

We also augment P+PT to monitor these metrics. We find that on some processors the algorithm can experience a higher rate of missing and duplicate eviction sets. At best, the algorithm missed 1% of the total eviction sets for the i7-6700K, also experiencing the highest rate of duplicates at 5%. At worst, 14% of the total eviction sets were missing for the i7-1100K. It may be possible that further tuning of the P+PT algorithm to the processors we use would improve the results. We leave testing this to future work.

7. Conclusion

We detail several enhancements to contention-based side-channel attacks by accelerating the initial generation of LLC eviction sets. Our work removes the uncertainty posed by the proprietary LLC slice function implemented in all major Intel processor families, which otherwise prolongs the running time of the eviction set generation procedure.

We design a technique for memory slice classification that uses weird gates, allowing us to determine slice mappings in unprivileged scenarios. Using knowledge of the reverse-engineered slice functions, we evaluate various methods for intra-page propagation of slice mappings, significantly reducing the time required to classify memory with our weird gate.

Finally, we present three optimisations for generating eviction sets for the entire LLC, leveraging knowledge of slice mappings to achieve a significant speedup in execution time over state-of-the-art.

Acknowledgements

This work was supported by the Air Force Office of Scientific Research (AFOSR) under award number FA9550-24-1-0079; the Alfred P. Sloan Research Fellowship; an ARC Discovery Project number DP210102670; the Defense Advanced Research Projects Agency (DARPA) under contract numbers W912CG-23-C-0022, Defence Science and Technology Group (DSTG), Australia under Agreement No. 11965; the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972; ISF grant no. 1807/23; Len Blavatnik and the Blavatnik Family Foundation; Stellar Development Foundation; and gifts from Cisco and Qualcomm.

The views and conclusions contained in this document are those of the authors and should not be interpreted as

representing the official policies, either expressed or implied, of the U.S. Government.

References

[1] B. B. Brumley and R. M. Hakala, "Cache-timing template attacks," in *AsiaCrypt*, 2009.

[2] C. Chuengsatiansup, D. Genkin, J. Kuepper, M. Wagner, D. Wu, and Y. Yarom, "0xADE1A1DE/AssemblyLine," 2022. [Online]. Available: https://github.com/0xADE1A1DE/AssemblyLine

[3] S. A. Crosby, D. S. Wallach, and R. H. Riedi, "Opportunities and limits of remote timing attacks," *ACM Trans. Inf. Syst. Secur.*, 2009. [Online]. Available: https://doi.org/10.1145/1455526.1455530

[4] G. Didier and C. Maurice, "Calibration done right: Noiseless Flush+Flush attacks," in *DIMVA*, 2021.

[5] C. Disselkoen, D. Kohlbrenner, L. Porter, and D. Tullsen, "Prime+Abort: A timer-free high-precision L3 cache attack using Intel TSX," in *USENIX Sec.*, 2017. [Online]. Available: https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/disselkoen

[6] J. Doweck, W.-F. Kao, A. K.-y. Lu, J. Mandelblat, A. Rahatekar, L. Rappoport, E. Rotem, A. Yasin, and A. Yoaz, "Inside 6th-generation Intel Core: New microarchitecture code-named Skylake," *IEEE Micro*, vol. 37, no. 2, 2017.

[7] D. Evtyushkin, T. Benjamin, J. Elwell, J. A. Eitel, A. Sapello, and A. Ghosh, "Computing with time: Microarchitectural weird machines," in *ASPLOS*, 2021.

[8] D. Genkin, L. Pachmanov, E. Tromer, and Y. Yarom, "Drive-by key-extraction cache attacks from portable code," in *ACNS*, 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-93387-0_5

[9] D. Genkin, W. Kosasih, F. Liu, A. Trikalinou, T. Unterluggauer, and Y. Yarom, "CacheFX: A framework for evaluating cache security," in *AsiaCCS*, 2023.

[10] L. Gerlach, S. Schwarz, N. Faroß, and M. Schwarz, "Efficient and generic microarchitectural hash-function recovery," in *IEEE SP*, 2023.

[11] D. Gruss, R. Spreitzer, and S. Mangard, "Cache template attacks: Automating attacks on inclusive last-level caches," in *USENIX Sec.*, 2015. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/gruss

[12] D. Gruss, C. Maurice, and S. Mangard, "Rowhammer.js: A remote software-induced fault attack in Javascript," in *DIMVA*, 2016.

[13] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, "Flush+Flush: A fast and stealthy cache attack," in *DIMVA*, 2016.

[14] G. Horowitz, E. Ronen, and Y. Yarom, "Spec-o-Scope: Cache probing at cache speed," in *CCS*, 2024.

[15] R. Hund, C. Willems, and T. Holz, "Practical timing side channel attacks against kernel space ASLR," in *IEEE SP*, 2013.

[16] M. S. Inci, B. Gulmezoglu, G. Irazoqui, T. Eisenbarth, and B. Sunar, "Seriously, get off my cloud! cross-VM RSA key recovery in a public cloud," 2015. [Online]. Available: https://eprint.iacr.org/2015/898

[17] Intel, "Intel 64 and IA-32 architectures software developer manuals," https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html.

[18] Intel, "12th generation Intel Core processors," https://edc.intel.com/content/www/us/en/design/ipla/software-development-platforms/client/platforms/alder-lake-desktop/12th-generation-intel-core-processors-datasheet-volume-1-of-2/010/.

[19] Intel, "6th generation Intel core processor family uncore performance monitoring reference manual," https://www.intel.com/content/www/us/en/content-details/671274/6th-generation-intel-core-processor-family-uncore-performance-monitoring-reference-manual.html, 2016.

[20] Intel, "Intel Xeon processor scalable memory family uncore performance monitoring reference manual," https://www.intel.com/content/www/us/en/content-details/671389/intel-xeon-processor-scalable-memory-family-uncore-performance-monitoring-reference-manual.html.

[21] G. Irazoqui, T. Eisenbarth, and B. Sunar, "S$A: A shared cache attack that works across cores and defies VM sandboxing – and its application to AES," in *IEEE SP*, 2015.

[22] G. Irazoqui, T. Eisenbarth, and B. Sunar, "Systematic reverse engineering of cache slice selection in Intel processors," in *DSD*, 2015.

[23] I. Kang, W. Wang, J. Kim, S. van Schaik, Y. Tobah, D. Genkin, A. Kwong, and Y. Yarom, "SledgeHammer: Amplifying Rowhammer via bank-level parallelism," in *USENIX Sec.*, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/kang

[24] D. A. Kaplan, "Optimization and amplification of cache side channel signals," arXiv 2303.00122, 2023. [Online]. Available: http://arxiv.org/abs/2303.00122

[25] H. Kario, "Out of the Box Testing," Cryptology ePrint Archive, Paper 2023/1441, 2023. [Online]. Available: https://eprint.iacr.org/2023/1441

[26] D. Katzman, W. Kosasih, C. Chuengsatiansup, E. Ronen, and Y. Yarom, "The gates of time: Improving cache attacks with transient execution," in *USENIX Sec.*, 2023. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/katzman

[27] T. Kessous, "Prune-PlumTree—finding-eviction-sets-at-scale," https://github.com/TomKessous/Prune-PlumTree---Finding-Eviction-Sets-at-Scale, 2023.

[28] T. Kessous and N. Gilboa, "Prune+PlumTree - finding eviction sets at scale," in *IEEE SP*, 2024.

[29] C. I. King, "Stress-Ng," https://github.com/ColinIanKing/stress-ng, 2024.

[30] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard,

T. Prescher, M. Schwarz, and Y. Yarom, "Spectre attacks: Exploiting speculative execution," in *IEEE SP*, 2019.

[31] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, A. Fogh, J. Horn, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," in *USENIX Sec.*, 2018. [Online]. Available: https://www.usenix.org/conference/usenixsecurity18/presentation/lipp

[32] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-level cache side-channel attacks are practical," in *IEEE SP*, 2015.

[33] C. Maurice, N. Le Scouarnec, C. Neumann, O. Heen, and A. Francillon, "Reverse engineering Intel last-level cache complex addressing using performance counters," in *RAID*, 2015.

[34] J. D. McCalpin, "Mapping addresses to L3/CHA slices in Intel processors," https://hdl.handle.net/2152/87595, 2021.

[35] B. Morgan, "BMorgan1296/Perfcounters," https://github.com/BMorgan1296/perfcounters, 2024.

[36] S. O'Connell, L. A. Sour, R. Magen, D. Genkin, Y. Oren, H. Shacham, and Y. Yarom, "Pixel thief: Exploiting SVG filter leakage in Firefox and Chrome," in *USENIX Sec.*, 2024. [Online]. Available: https://www.usenix.org/conference/usenixsecurity24/presentation/oconnell

[37] Y. Oren, V. P. Kemerlis, S. Sethumadhavan, and A. D. Keromytis, "The spy in the sandbox: Practical cache attacks in JavaScript and their implications," in *CCS*, 2015.

[38] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: The case of AES," IACR ePrint archive 2005/271, 2005. [Online]. Available: https://eprint.iacr.org/2005/271

[39] R. Paccagnella, L. Luo, and C. W. Fletcher, "Lord of the ring(s): Side channel attacks on the CPU on-chip ring interconnect are practical," in *USENIX Sec.*, 2021. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/paccagnella

[40] C. Percival, "Cache missing for fun and profit," in *BSDCan*, 2005. [Online]. Available: https://www.daemonology.net/papers/htt.pdf

[41] A. Purnal, L. Giner, D. Gruss, and I. Verbauwhede, "Systematic analysis of randomization-based protected cache architectures," in *SP*, 2021.

[42] A. Purnal, F. Turan, and I. Verbauwhede, "Prime+Scope: Overcoming the observer effect for high-precision cache contention attacks," in *CCS*, 2021.

[43] A. Purnal, M. Bognar, F. Piessens, and I. Verbauwhede, "ShowTime: Amplifying arbitrary CPU timing side channels," in *AsiaCCS*, 2023.

[44] J. R. Quinlan, "Induction of decision trees," in *Machine Learning*, 1986.

[45] M. K. Qureshi, "CEASER: mitigating conflict-based cache attacks via encrypted-address and remapping," in *MICRO*, 2018.

[46] A. Shusterman, A. Agarwal, S. O'Connell, D. Genkin, Y. Oren, and Y. Yarom, "Prime+Probe 1, JavaScript 0: Overcoming browser-based side-channel defenses," in *USENIX Sec.*, 2021. [Online]. Available: https://www.usenix.org/conference/usenixsecurity21/presentation/shusterman

[47] W. Song and P. Liu, "Dynamically finding minimal eviction sets can be quicker than you think for side-channel attacks against the LLC," in *RAID*, 2019. [Online]. Available: https://www.usenix.org/conference/raid2019/presentation/song

[48] J. P. Thoma and T. Güneysu, "Write me and I'll tell you secrets - write-after-write effects on Intel CPUs," in *RAID*, 2022.

[49] Y. Tsunoo, E. Tsujihara, K. Minematsu, and H. Miyauchi, "Cryptanalysis of block ciphers implemented on computers with cache," in *ISITA*, 2002.

[50] Y. Tsunoo, T. Saito, T. Suzaki, M. Shigeri, and H. Miyauchi, "Cryptanalysis of DES implemented on computers with cache," in *CHES*, 2003.

[51] M. Vanhoef and E. Ronen, "Dragonblood: Analyzing the Dragonfly handshake of WPA3 and EAP-pwd," in *IEEE SP*, 2020.

[52] P. Vila, B. Köpf, and J. F. Morales, "Theory and practice of finding eviction sets," in *IEEE SP*, 2019.

[53] Vimalm and J. D. McCalpin, "Re: Turning off the cache coherence directory system wide in Intel Xeon Gold 6242 processor," 2022. [Online]. Available: https://community.intel.com/t5/Processors/Turning-off-the-cache-coherence-directory-system-wide-in-Intel/m-p/1364445#M56606

[54] P.-L. Wang, R. Paccagnella, R. S. Wahby, and F. Brown, "Bending microarchitectural weird machines towards practicality," in *USENIX Sec.*, 2024.

[55] M. Werner, T. Unterluggauer, L. Giner, M. Schwarz, D. Gruss, and S. Mangard, "ScatterCache: Thwarting cache attacks via cache set randomization," in *USENIX Sec.*, 2019.

[56] Z. Wu, Z. Xu, and H. Wang, "Whispers in the hyper-space: High-speed covert channel attacks in the cloud," in *USENIX Sec.*, 2012.

[57] Z. Xue, J. Han, and W. Song, "CTPP: A fast and stealth algorithm for searching eviction sets on Intel processors," in *RAID*, 2023.

[58] M. Yan, R. Sprabery, B. Gopireddy, C. Fletcher, R. Campbell, and J. Torrellas, "Attack directories, not caches: Side channel attacks in a non-inclusive world," in *IEEE SP*, 2019.

[59] M. Yan, C. W. Fletcher, and J. Torrellas, "Cache telepathy: Leveraging shared resource attacks to learn DNN architectures," in *USENIX Sec.*, 2020. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/yan

[60] Y. Yarom and K. Falkner, "Flush+Reload: A high resolution, low noise, L3 cache side-channel attack," in *USENIX Sec.*, 2014. [Online]. Available: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/yarom

[61] Y. Yarom, Q. Ge, F. Liu, R. B. Lee, and G. Heiser, "Mapping the Intel last-level cache," IACR ePrint 2015/905, 2015. [Online]. Available: https://eprint.iacr.org/2015/905

[62] Y. A. Younis, K. Kifayat, Q. Shi, and B. Askwith, "A new prime and probe cache side-channel attack for cloud computing," in *CIT*, 2015.

[63] Y. Yuan, Q. Pang, and S. Wang, "Automated side channel analysis of media software with manifold learning," in *USENIX Sec.*, 2022. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/yuan-yuanyuan

[64] Z. N. Zhao, A. Morrison, C. W. Fletcher, and J. Torrellas, "Last-level cache side-channel attacks are feasible in the modern public cloud," in *ASPLOS*, 2024.

Appendix A. Comparator Gate Code

Listing 1 shows the assembly code for the comparator gate, which is used to compare memory latencies for two cache lines (`input` and `compare`) in the LLC. The code is just-in-time compiled using AssemblyLine [2] for ease of use. This is the main building block of our userspace slice index prediction algorithm.

Running a Single Measurement. To carry out a single measurement using the comparator gate:

- **Setup:** Place the `input`, `compare` and `signal` cache lines into the LLC using L2 eviction sets.
- **Lines 1–11:** Halt speculation with memory fences, place `signal` in R10 for later use as RDTSCP will overwrite RDX.
- **Lines 19–22:** Set up a speculative window with return-based gadget [24], squashing the transient branch when the request for `input` in RDI completes.
- **Lines 13–15:** Access `compare` in RSI, followed by a hand-tuned fixed-length delay chain of instructions as in [26], to prevent the processor from accessing `signal` before the `compare` cache line arrives.
- **Lines 16–17:** Access `signal` in R10 after the chain and halt speculation with a load fence.
- **Lines 24–35:** Determine whether `signal` is cached in the L1 or not using the RDTSCP timer instruction.

Listing 1: Comparator Gate Assembly Code

```
1  xor rdi, 0x800
2  mov rax, [rdi] ; Cache *input's page in TLB
3  xor rdi, 0x800
4  xor rsi, 0x800
5  mov rax, [rsi] ; Cache *compare's page in TLB
6  xor rsi, 0x800
7  mov r10, rdx
8  mfence
9  lfence
10 ; DC_LEN sets the delay chain length
11 call long 0x9 + (DC_LEN * 0x3) ; Call L19
12
13 mov rsi, [rsi] ; Access *compare
14 add r10, rsi
15 ...            ; Hand-tuned delay chain
16 mov r10, [r10] ; Access *signal
17 lfence         ; Halt speculation
18
19 mov rax, 0x16 + (DC_LEN * 0x3) ; Offset to L24
20 add rax, [rdi] ; Access *input
21 add [rsp], rax ; Set return address to L24
22 ret            ; Mispredict to L13
23
24 lfence         ; Halt speculation
25 xor r10, 0x800
26 mov rax, [r10] ; Cache *signal's page in TLB
27 xor r10, 0x800
28 mfence
29 lfence
30 rdtscp
31 mov r9, rax
32 add r10, [r10] ; Measure access to *signal
33 rdtscp
34 sub rax, r9
35 ret            ; Return access time
```

TABLE 6: Recovery methods for the Intel LLC function.

Work	Slice Recovery	Function Recovery	Retrieval Time	
			Linear	Non-Linear
Hund et al. [15]	Cache Conflict	Manual	Manual	–
Irazoqui et al. [22]	Cache Conflict	Linear Eq.	Manual	–
Yarom et al. [61]	Access Timing	Manual	–	Manual
Inci et al. [16]	Cache Conflict	Manual	–	Manual
McCalpin [34]	Perf. Counter	Manual	Manual	Manual
Maurice et al. [33]	Perf. Counter	Automated Grey-box	Minutes	–
Gerlach et al. [10]	Perf. Counter, Access Timing	Automated Black-box	Minutes	Weeks
Ours	Perf. Counter	Direct physical address access	∼25 ms	<1000 ms

Appendix B. Generic Slice Function Retrieval

To build the decision tree and improve our slice predictions, we need to first recover the slice function in a generic and time-efficient manner for any modern Intel processor. In our threat model, the attacker obtains the targeted processor and recovers the function offline. This only needs to be completed once per processor.

Slice Function Recovery Methods. Table 6 details prior methods for reverse engineering the slice function, comparing the method of slice recovery for an address, the entire function recovery, and time taken. The majority of prior techniques are manual, and none beside work by Gerlach et al. [10] can automatically recover the function in linear and non-linear cases. We aim to improve on this by providing an efficient and automated method for both linear and non-linear functions. Our codebase details the specifics of the implementation, but we provide a high-level overview here.

Determining Memory Slice Index Using Performance Counters. To retrieve any information regarding the slice function, the first step is to identify which slice a memory address maps to.

TABLE 7: Recovery times for slice functions. Non-linear take longer due to the sequence-centric recovery approach.

Processor	Slices	Linear	Time (ms)
i7-6700K	4	✓	25
i7-11700KF	8	✓	19
i7-13700H	8	✓	24
i7-8700	6	✗	291
i7-9850H	6	✗	225
i7-10710U	6	✗	190
i9-12900KF	10	✗	522
i9-13900KF	12	✗	740
i9-14900K	12	✗	930

The known reverse engineering process requires finding two memory addresses differing by a single physical bit to recover slice information [33, 34]. This approach requires searching for pairs of addresses in large buffers of memory and is limited to the currently installed system RAM.

We implement a custom kernel module which bypasses memory controller checks, enabling read requests for any physical address of our choosing without the need for a large buffer of memory. By directly accessing any physical memory address, we can reconstruct the slice function up to the memory limit of the processor.

We likewise use the CLFLUSH instruction to generate read requests and observe slice lookups [33, 34]. To interface with the uncore performance counters [17, 19], we employ a custom library [35] and monitor read lookups[4] on each available slice. By repeatedly issuing CLFLUSH commands on an address, we can identify the slice to which an address maps by observing the performance counter with the highest number of read requests.

TABLE 8: Recovered slice functions for various Intel Core processor generations with a power of two slice count. The XOR permutation masks m are used to determine each slice index bit.

Year	Generation	Slices	Permutation Masks
2017	7	2	$m_0 = \texttt{0x5b5f575440}$
2013, 2015, 2017	4–7	4	$m_0 = \texttt{0x5b5f575440}$ $m_1 = \texttt{0x6eb5faa880}$
2021, 2023	11, 13	8	$m_0 = \texttt{0x5b5f575440}$ $m_1 = \texttt{0x71aeeb1200}$ $m_2 = \texttt{0x06d87f2c00}$

B.1. Linear Functions.

Linear slice functions use a basic XOR operation with permutation masks to select address bits for calculating the slice index. To recover the slice function, we analyse the slices for physical addresses `0x0`, `0x40`, `0x80`, and so forth, up to the maximum physical address supported by the processor. We start with `0x40` instead of `0x0` because the lower bits (cache line offset bits) are not involved in the slice function. For instance, let H denote the unknown slice function and the recovered slice index as s, we have $s = H(\texttt{0x0}) \bigoplus H(\texttt{0x40})$. We then decompose s bit-wise and map it across the permutation masks array m at index $\log_2(\texttt{0x40}) = 6$. By recording the slice index for each tested physical address and mapping the bits of

4. Using `UNC_CBO_CACHE_LOOKUP.ANY_MESI` perf counter.

TABLE 9: Recovered slice functions for various Intel Core processor generations with a non-power of two slice count. The XOR permutation masks m calculate each index bit for a lookup into the base sequences shown in Table 10.

Year	Generation	Slices	Permutation Masks
2018, 2019, 2020, 2021	8, 9, 10, 12	6, 10	$m_0 = \texttt{0x21ae7be000}$ $m_1 = \texttt{0x435cf7c000}$ $m_2 = \texttt{0x2717946000}$ $m_3 = \texttt{0x4e2f28c000}$ $m_4 = \texttt{0x1c5e518000}$ $m_5 = \texttt{0x38bca30000}$ $m_6 = \texttt{0x50d73de000}$
2022	13	12	$m_0 = \texttt{0x52c6a78000}$ $m_1 = \texttt{0x30342b8000}$ $m_2 = \texttt{0x547f480000}$ $m_3 = \texttt{0x3d47f48000}$ $m_4 = \texttt{0x1c5e518000}$ $m_5 = \texttt{0x38bca30000}$ $m_6 = \texttt{0x23bfe18000}$ $m_7 = \texttt{0x0000000000}$ $m_8 = \texttt{0x7368dc0000}$
2023	14	12	$m_0 = \texttt{0x2f52c6a78000}$ $m_1 = \texttt{0x0cb0342b8000}$ $m_2 = \texttt{0x35d47f480000}$ $m_3 = \texttt{0x39bd47f48000}$ $m_4 = \texttt{0x109c5e518000}$ $m_5 = \texttt{0x2038bca30000}$ $m_6 = \texttt{0x0e23bfe18000}$ $m_7 = \texttt{0x000000000000}$ $m_8 = \texttt{0x31f368dc0000}$

TABLE 10: Intel Core processor generations with non-power-of-two slices and their corresponding base sequences.

Year	Generation	Slices	Base Sequence
2018, 2019	8, 9	6	0123 1434 1032 0525 1032 0525 0525 1434 0123 5052 5052 4143 1032 4143 4143 5052 2301 5250 3210 4341 3210 4341 4341 5250 2301 3414 3414 2505 3210 2505 2505 3414
2020, 2021	10, 12	10	0505 3636 1414 2727 1414 2727 0589 3698 4141 7272 5098 6389 5050 6363 4189 7298 4141 7272 5050 6363 5050 6363 8941 9872 0505 3636 9814 8927 1414 2727 8905 9836
2022, 2023	13, 14	12	0145 1858 3276 2b6b 1054 0949 2b6b 3a7a 2367 b2b6 9094 8185 3276 a3a7 8185 9094 6723 b6b2 5410 8581 7632 a7a3 8581 9490 4501 5818 7a3a 6b2b 5410 4909 6b2b 7a3a 6b2b 7632 5818 4501 7a3a 6723 4909 5818 8581 5410 b6b2 a7a3 9490 4501 a7a3 b6b2 8185 1054 b2b6 2367 9094 0145 a3a7 b2b6 2b6b 3276 1858 0949 3a7a 2367 0949 1858 3a7a 2367 0949 1054 2b6b 3a7a 1858 0145 9094 8185 a3a7 3276 8185 9094 b2b6 2367 9490 4501 a7a3 7632 8581 9490 b6b2 6723 7a3a 6b2b 4909 5410 6b2b 7a3a 5818 4501 5410 4909 6723 7a3a 4909 5818 7632 6b2b b6b2 a7a3 4501 9490 a7a3 b6b2 5410 8581 3276 a3a7 0145 9094 a3a7 b2b6 1054 8185 1858 0949 2367 3a7a 0949 1858 3276 2b6b

s to the permutation masks array m, we can reconstruct the permutation masks bit by bit. Repeating this process for each significant bit allows us to fully recover the slice function used by the processor.

B.2. Non-Linear Functions.

In contrast, recovering non-linear functions is inherently more complex. To address this, we develop a method to simultaneously recover both the permutation masks and the base sequence. Initially, we do not know the length of the sequence, so we start with an initial guess of length 1. We measure slice mappings from address `0x0` to establish the initial base sequence B, again using performance counters and the kernel module. We then generate a temporary sequence T of slice mappings for the next address bit, comparing these mappings to our guessed base sequence by searching for the XOR permutation used. Specifically, we search for the XOR permutation x where $T \bigoplus x = B$. If we find a matching x, then we split this bit-wise into the permutation masks array m and continue to the next significant address bit, the same method as the linear function. If we do not find any matching XOR permutations, we double the sequence length and repeat the process. For the new length, we measure slice mappings from $0 \dots 2^{n-1}$, where n is the current guessed sequence length. By iterating through this process—starting with a small sequence length and progressively increasing it until the correct length is found, we can reconstruct the permutation masks and the base sequence.

Table 7 displays the time taken to extract the slice function on several generations of processors. As can be seen, our automated recovery approach can retrieve this in under 30ms for linear and under a second for non-linear.

B.3. Recovered Slice Functions

Tables 8 and 9 detail the recovered slice functions for various Intel Core processor generations. Table 10 shows the base sequences for non-power-of-two slice counts.

Appendix C. Meta-Review

The following meta-review was prepared by the program committee for the 2025 IEEE Symposium on Security and Privacy (S&P) as part of the review process as detailed in the call for papers.

C.1. Summary

This paper provides a number of techniques that jointly significantly improve the performance of generating last-level cache eviction sets. The paper first proposes a way to identify the slice that an address maps to by using a "weird gate" that performs a timing comparison. The paper also proposes methods for intra-page propagation of slice mappings. Based on these new proposals, the paper in the end succeeds in speeding up the eviction set generation for the full last-level-cache.

C.2. Scientific Contributions

- Creates a New Tool to Enable Future Science
- Addresses a Long-Known Issue
- Provides a Valuable Step Forward in an Established Field
- Establishes a New Research Direction

C.3. Reasons for Acceptance

1) This paper creates a new tool to generate certain eviction sets. This helps to assess the security of CPU microarchitecture designs.
2) This paper addresses a long-known issue, in that identification of eviction sets is a common prerequisite for cache-based attacks.
3) This paper provides a valuable step forward by significantly increasing the efficiency with which eviction sets can be identified.
4) This paper establishes a new research direction as the PC found some of the slice+slice technique to be novel.

C.4. Noteworthy Concerns

1) The PC found this paper to be focused very deeply on a single aspect of a single vendor's CPU design. While the paper does a nice job building an effective eviction set identification tool out of access latency variations, it is ultimately a reactive piece of work focused on specific CPU makes and models.